Practical 2 – Querying shopdb — joins, subqueries, aggregates and grouping.

```
USE shopdb;
-- 1. List each order with the customer's name and the order date.
select
        o.order_id,
        c.name,
        o.order_date
from customers c
inner join orders o
        on c.customer_id=o.customer_id;
```

OUTPUT:

order_id	name	order_date
1	Asha Rao	2024-04-01
2	Ravi Nair	2024-05-03
3	Asha Rao	2024-06-10
4	Sara Ali	2024-06-12

```
-- 2. List every customer with their number of orders, including customers who have none.
select
        c.customer_id,
        c.name,
        count(o.order_id) as number_of_orders
from customers c
left join orders o
        on c.customer_id=o.customer_id
group by
        c.customer_id,
        c.name;
```

OUTPUT:

customer_id	name	number_of_orders
1	Asha Rao	2
2	Ravi Nair	1
3	Sara Ali	1
4	Om Verma	0
5	Neha Shah	0

-- 3. List all products priced above the average product price.

```
select *
from products
where price>(
        select avg(price)
        from products
);
```

OUTPUT:

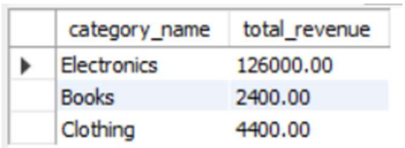


	product_id	product_name	category_id	price	stock
▶	1	Laptop	1	60000.00	9
*	NULL	NULL	NULL	NULL	NULL

-- 4. Show the total revenue per category, excluding cancelled orders. (Revenue of a line = quantity × unit_price.)

```
select
        c.category_name,
        SUM(oi.quantity * oi.unit_price) AS total_revenue
from categories c
join products p
        on c.category_id = p.category_id
join order_items oi
        on p.product_id = oi.product_id
join orders o
        on oi.order_id = o.order_id
where o.status <> 'Cancelled'
group by
        c.category_id,
        c.category_name;
```

OUTPUT:



	category_name	total_revenue
▶	Electronics	126000.00
	Books	2400.00
	Clothing	4400.00

-- 5. Show only the categories whose total revenue exceeds ₹5,000.

```
select
        c.category_name,
        sum(oi.quantity * oi.unit_price) as total_revenue
from categories c
```

```
join products p
        on c.category_id = p.category_id
join order_items oi
        on oi.product_id = p.product_id
join orders o
        on o.order_id=oi.order_id
where o.status <> "Cancelled"
group by
        c.category_id,
        c.category_name
having sum(oi.quantity * oi.unit_price)>5000;
```

OUTPUT:

	category_name	total_revenue
▶	Electronics	126000.00

```
-- 6. List products priced above the average price within their own category.
select
        p.product_id,
        p.product_name,
        p.price,
        (
                select round(avg(p2.price),2)
                from products p2
                where p2.category_id = p.category_id
        ) as average_category_price
from products p
where p.price > (
        select avg(p2.price)
        from products p2
        where p2.category_id = p.category_id
);
```

OUTPUT:

	product_id	product_name	price	average_category_price
▶	1	Laptop	60000.00	31000.00
	3	SQL Book	800.00	533.33
	6	Jacket	3000.00	1850.00